

Sistemi e Architetture per Big Data - A.A. 2025/26

Progetto 2: Analisi in tempo reale di dati sui voli con Apache Flink

Docenti: Valeria Cardellini, Matteo Nardelli
Dipartimento di Ingegneria Civile e Ingegneria Informatica
Università degli Studi di Roma Tor Vergata

Background

Il monitoraggio operativo del trasporto aereo richiede l'analisi continua di grandi volumi di eventi relativi ai voli. Informazioni quali ritardi in partenza, ritardi in arrivo, cancellazioni, deviazioni e aeroporti coinvolti consentono di identificare tempestivamente situazioni di congestione, rotte critiche e compagnie o aeroporti caratterizzati da condizioni operative anomale. A differenza dell'analisi batch, in cui l'intero dataset storico viene processato a posteriori, l'analisi in tempo reale richiede che gli eventi vengano elaborati man mano che sono prodotti, mantenendo aggregazioni aggiornate su finestre temporali e gestendo possibili ritardi o disordine nell'arrivo degli eventi.

Requisiti del progetto

Lo scopo del progetto è rispondere ad alcune query su uno stream di eventi relativi ai voli aerei operati negli Stati Uniti, utilizzando l'approccio di processamento a *stream* con Apache Flink¹. In particolare, si richiede di sviluppare un'applicazione Flink in grado di ricevere progressivamente gli eventi di volo, assegnare a ciascun evento un timestamp logico (*event time*), calcolare statistiche su finestre temporali e produrre risultati aggiornati in formato CSV.

Dataset

Il dataset di riferimento è lo stesso usato per il Progetto 1, fornito dal Dipartimento dei trasporti degli Stati Uniti d'America² e disponibile al seguente URL³: <http://www.ce.uniroma2.it/courses/sabd2526/project/project-1-data.tar.gz>.

Il dataset contiene circa 2.200.000 eventi, che descrivono i voli effettuati nel periodo dal 1 gennaio 2025 al 30 aprile 2025. Ogni evento descrive un singolo volo ed è caratterizzato, tra gli altri, dai seguenti campi:

- **Compagnia aerea** (OP_UNIQUE_CARRIER);
- **Data del volo**, comprensiva di anno, mese e giorno (YEAR, MONTH, DAY_OF_MONTH);
- **Aeroporto di partenza e di arrivo** (ORIGIN_AIRPORT_ID, DEST_AIRPORT_ID);

¹<https://flink.apache.org/>

²<https://www.transtats.bts.gov/>

³shasum del file compresso in TAR GZ: 17be276b72bd987e72598b0ea4907c3b19350606

- **Orario schedulato di partenza** (CRS_DEP_TIME);
- **Ritardo in partenza e in arrivo** (DEP_DELAY, ARR_DELAY), espressi in minuti; possono assumere anche valori negativi (voli in anticipo);
- **Stato del volo**, inclusa l'informazione se il volo è stato cancellato (CANCELLED) o deviato (DIVERTED);
- **Cause del ritardo**, tra cui ritardi dovuti alla compagnia, alle condizioni meteo, al sistema aeroportuale, alla sicurezza e al ritardo dell'aeromobile precedente (CARRIER_DELAY, WEATHER_DELAY, NAS_DELAY, SECURITY_DELAY, LATE_AIRCRAFT_DELAY).

Il dataset fornito è di natura storica. Ai fini del progetto, deve essere trattato come uno stream mediante una fase di replay degli eventi in tempo reale.

Gli eventi devono essere emessi seguendo l'ordine temporale determinato dalla data del volo e dall'orario schedulato di partenza, riordinando pertanto la sequenza degli eventi rispetto all'ordine in cui sono presenti nel dataset fornito. L'*event time* di ciascun evento deve essere ricavato combinando i campi YEAR, MONTH, DAY_OF_MONTH e CRS_DEP_TIME. La generazione degli eventi durante il replay deve inoltre prevedere un'accelerazione temporale rispetto all'*event time* originale, mantenendo inalterate le relazioni temporali tra gli eventi mediante una compressione uniforme della scala temporale. Il fattore di accelerazione adottato deve essere esplicitato e motivato nella relazione.

Nel caso in cui l'applicazione introduca eventi fuori ordine, è necessario adottare una strategia di watermarking e motivare nella relazione la scelta del ritardo massimo ammesso.

La fase di data ingestion può essere implementata usando Apache Kafka, Apache Pulsar, Apache NiFi oppure tramite un file stream. In ogni caso, la relazione deve descrivere in modo chiaro il meccanismo adottato per la produzione, l'ordinamento e il consumo degli eventi da parte dell'applicazione di stream processing.

I valori mancanti dei campi numerici possono essere trattati come `null` oppure come zero; la scelta adottata deve essere motivata nella relazione. Nel calcolo delle statistiche sui ritardi, salvo ove diversamente specificato, considerare esclusivamente i voli non cancellati e non deviati.

Query

Ogni elemento dello stream corrisponde a un singolo volo. Si richiede di sviluppare un'applicazione di processamento in streaming che risponda alle seguenti query.

Q1 La query Q1 si concentra sul monitoraggio in tempo reale dello stato operativo delle principali compagnie aeree. Facendo riferimento ai voli relativi alle compagnie AA (American Airlines), DL (Delta), UA (United) e WN (Southwest), aggregare gli eventi usando finestre tumbling di durata pari a 1 ora, basate sull'*event time*.

Per ciascuna finestra e per ciascuna compagnia, calcolare:

- il numero totale di voli osservati;
- il numero di voli completati (cioè non cancellati e non deviati), cancellati e deviati;
- il valor medio di DEP_DELAY, considerando solo i voli non cancellati;
- il tasso di cancellazione, definito come percentuale di voli cancellati sul totale dei voli osservati nella finestra;

- il tasso di partenze in ritardo, definito come percentuale di voli non cancellati con `DEP_DELAY` maggiore di 15 minuti.

L'output della query deve avere il seguente schema:

```
window_start, window_end, airline, num_flights, completed, cancelled,
diverted, dep_delay_mean, cancellation_rate, late_departure_rate
```

ESEMPIO DI OUTPUT

```
# window_start, window_end, airline, num_flights, completed, cancelled, diverted,
# dep_delay_mean, cancellation_rate, late_departure_rate
2025-01-01 08:00:00, 2025-01-01 09:00:00, AA, 124, 118, 3, 3, 11.42, 2.42, 23.73
2025-01-01 08:00:00, 2025-01-01 09:00:00, DL, 117, 113, 1, 3, 8.95, 0.85, 18.58
```

(i valori indicati non sono significativi rispetto all'output atteso)

Q2 La query Q2 si concentra sull'identificazione, in tempo reale, degli aeroporti di partenza maggiormente interessati da ritardi significativi.

Considerare tutti i voli nello stream e aggregare gli eventi per aeroporto di partenza (`ORIGIN_AIRPORT_ID`) usando finestre temporali basate sull'*event time*. Si consideri come *ritardo significativo in partenza* un volo non cancellato e non deviato con `DEP_DELAY` maggiore di 30 minuti.

Calcolare la query sulle seguenti finestre temporali:

- 1 ora (event time);
- 6 ore (event time);
- dall'inizio del dataset.

Per ciascuna finestra, calcolare la classifica aggiornata in tempo reale dei primi 10 aeroporti di partenza ordinati per numero decrescente di voli con ritardo significativo in partenza.

Per ogni aeroporto in classifica, riportare:

- il numero totale di voli osservati nella finestra;
- il numero di voli con ritardo significativo in partenza;
- il valor medio di `DEP_DELAY`, considerando solo i voli non cancellati e non deviati;
- il massimo valore di `DEP_DELAY` osservato nella finestra;
- una lista dei voli con ritardo significativo in partenza, riportando almeno compagnia aerea, aeroporto di destinazione e valore di `DEP_DELAY`. **Nota:** Poiché la lista potrebbe essere lunga, si chiede di riportare al più i primi 20 voli con ritardo significativo, ordinati per `DEP_DELAY` decrescente.

Si considerino, per ciascuna finestra, solo gli aeroporti con almeno 30 voli non cancellati e non deviati.

L'output della query deve avere il seguente schema:

```
ts, rank, origin_airport_id, num_flights, severe_delays, dep_delay_mean,
dep_delay_max, delayed_flights
```

dove:

- `ts`: timestamp relativo all'inizio della finestra su cui è stata calcolata la classifica;
- `rank`: posizione dell'aeroporto nella classifica top-10;
- `origin_airport_id`: identificativo dell'aeroporto di partenza;
- `num_flights`: numero di voli non cancellati e non deviati osservati nella finestra;
- `severe_delays`: numero di voli con ritardo significativo in partenza;
- `dep_delay_mean`: ritardo medio in partenza dei voli considerati;
- `dep_delay_max`: massimo ritardo in partenza osservato nella finestra;
- `delayed_flights`: lista sintetica dei voli con ritardo significativo, ad esempio nella forma `[(carrier, dest, delay), ...]`.

ESEMPIO DI OUTPUT

```
# ts, rank, origin_airport_id, num_flights, severe_delays,  
# dep_delay_mean, dep_delay_max, delayed_flights  
  
2025-01-03 12:00:00, 1, 10397, 298, 17, 18.42, 214.00,  
[(AA,11298,214.00), (DL,13930,176.00), ...]  
2025-01-03 12:00:00, 2, 11292, 271, 14, 16.87, 188.00,  
[(UA,12892,188.00), (WN,14107,165.00), ...]
```

(i valori indicati non sono significativi rispetto all'output atteso)

Q3 La query Q3 si concentra sul calcolo in tempo reale della distribuzione dei ritardi in partenza per compagnia aerea e fascia oraria.

Facendo riferimento ai voli relativi alle compagnie AA (American Airlines), DL (Delta), UA (United) e WN (Southwest), considerare i valori di `DEP_DELAY` dei voli non cancellati e non deviati. Per ciascun volo, ricavare la fascia oraria di partenza programmata a partire dal campo `CRS_DEP_TIME`, considerando le 24 fasce orarie giornaliere.

Per ciascuna compagnia e per ciascuna fascia oraria, calcolare:

- il numero di voli considerati;
- il minimo di `DEP_DELAY`;
- il 25-esimo percentile;
- il 50-esimo percentile, o mediana;
- il 75-esimo percentile;
- il 90-esimo percentile;
- il massimo di `DEP_DELAY`.

Calcolare la query sulle seguenti finestre temporali:

- 1 giorno (event time);

- 7 giorni (event time);
- dall'inizio del dataset.

I percentili devono essere calcolati in tempo reale, senza ordinare tutti i valori e possibilmente senza accumularli interamente. Si utilizzi pertanto un algoritmo approssimato che consente di calcolare i percentili riducendo la quantità di memoria occupata al prezzo di una minore accuratezza, e.g., [1, 2, 3, 4, 5, 6]. La relazione deve descrivere brevemente l'algoritmo utilizzato.

L'output della query deve avere il seguente schema:

```
ts, airline, hour, count, min, p25, p50, p75, p90, max
```

dove:

- `ts`: timestamp relativo all'inizio della finestra su cui sono calcolate le statistiche;
- `airline`: compagnia aerea;
- `hour`: fascia oraria di partenza programmata, ricavata da `CRS_DEP_TIME`;
- `count`: numero di voli considerati;
- `min`: valore minimo del ritardo in partenza;
- `p25, p50, p75, p90`: percentili approssimati del ritardo in partenza;
- `max`: valore massimo del ritardo in partenza.

ESEMPIO DI OUTPUT

```
# ts, airline, hour, count, min, p25, p50, p75, p90, max
2025-01-01 00:00:00, AA, 8, 421, -12.00, -3.00, 4.00, 18.00, 47.00, 214.00
2025-01-01 00:00:00, DL, 8, 398, -15.00, -4.00, 3.00, 15.00, 39.00, 176.00
...
```

(i valori indicati non sono significativi rispetto all'output atteso)

Il risultato di ciascuna query deve essere consegnato in formato CSV. Si chiede inoltre di valutare sperimentalmente la latenza di processing, il throughput e l'eventuale impatto del parallelismo dell'applicazione, riportando l'analisi nella relazione e nella presentazione.

Composizione dei gruppi

Il progetto è dimensionato per un gruppo composto da **2 studenti**.

Per gruppi composti da 1 studente: si richiede di rispondere alle query Q1 e Q2. La gestione completa della pipeline di ingestion tramite Kafka, Pulsar o NiFi è opzionale; è tuttavia necessario simulare lo stream di input e scrivere i risultati in formato CSV.

Per gruppi composti da 3 studenti: in aggiunta ai requisiti sopra elencati, si richiede di implementare un meccanismo di rilevamento di eventi fuori ordine e di confrontare almeno due configurazioni di watermarking, discutendo l'impatto su completezza dei risultati e latenza. In alternativa, è possibile implementare la soluzione delle query sia in Apache Flink sia in Spark Streaming e confrontarne le prestazioni sulla stessa piattaforma.

Opzionale 1: realizzare una dashboard in Grafana (o strumento equivalente) per visualizzare l'andamento temporale delle metriche prodotte dalle query.

Opzionale 2: implementare la gestione della tolleranza ai guasti degli operatori con stato mediante il meccanismo di checkpointing di Apache Flink e valutarne l'impatto sulle prestazioni. La relazione dovrà descrivere la configurazione adottata e le modalità di verifica del corretto recupero dello stato a seguito di un guasto.

Svolgimento e consegna del progetto

Per ogni comunicazione via email è necessario specificare *[SABD]* nell'oggetto dell'email.

Comunicare la composizione del gruppo ai docenti entro **venerdì 19 giugno 2026** (solo se diversa rispetto al progetto 1). Il progetto è valido **solo** per l'A.A. 2025/26 ed il codice deve essere consegnato **entro martedì 7 luglio 2026**.

La consegna del progetto consiste in:

1. link a spazio di Cloud storage o repository contenente il codice del progetto da comunicare via email ai docenti entro la scadenza; inserire i risultati delle query in formato CSV in una cartella denominata `Results`;
2. relazione di lunghezza compresa tra le 4 e le 8 pagine, da inserire all'interno della cartella denominata `Report`; per la relazione si consiglia di usare il formato ACM proceedings (<https://www.acm.org>) oppure il formato IEEE proceedings (<https://www.ieee.org>); includere nella relazione lo schema dell'architettura di sistema utilizzata e l'analisi dei risultati sperimentali.
3. slide della presentazione orale, da inviare via email ai docenti **dopo** lo svolgimento della presentazione.

La presentazione si terrà tra **venerdì 10 e mercoledì 15 luglio 2026** (da confermare); ciascun gruppo avrà a disposizione **massimo 15 minuti** per presentare la propria soluzione (20 minuti per gruppi composti da 3 persone).

Valutazione del progetto

I principali criteri di valutazione del progetto saranno:

1. rispondenza ai requisiti;
2. correttezza delle query e gestione dell'event time;
3. architettura del sistema e deployment;
4. organizzazione del codice;
5. efficienza e analisi sperimentale;
6. organizzazione, chiarezza e rispetto dei tempi della presentazione orale.

Utilizzo di strumenti di Intelligenza Artificiale Generativa

L'utilizzo di strumenti di Intelligenza Artificiale Generativa è parzialmente consentito nell'ambito del progetto, purché avvenga in modo trasparente e consapevole e non sostitutivo del lavoro degli studenti.

Insieme al report finale, ciascun gruppo dovrà allegare una dichiarazione che specifichi:

- se sono stati utilizzati strumenti di IA generativa ed eventualmente quali;
- per quali attività sono stati impiegati (ad esempio: brainstorming, progettazione, scrittura del report, generazione di codice, debugging, documentazione, ecc.);
- quali parti del report, del codice o di altri artefatti del progetto sono state generate o significativamente modificate con il supporto di tali strumenti;
- quali contenuti sono stati successivamente revisionati, modificati o validati dagli studenti.

L'uso di strumenti di IA per il solo miglioramento linguistico o stilistico di contenuti originali prodotti dagli studenti non richiede una descrizione dettagliata, purché tali strumenti non introducano nuove idee, analisi, risultati o contenuti tecnici.

Durante la presentazione orale, tutti i componenti del gruppo dovranno essere in grado di discutere e giustificare le scelte progettuali, metodologiche e implementative adottate nel progetto, indipendentemente dall'eventuale utilizzo di strumenti di IA. I docenti potranno inoltre richiedere chiarimenti sul codice sviluppato, sulle soluzioni implementate e sul contributo effettivo degli studenti alle diverse parti del progetto.

La valutazione terrà conto della qualità del lavoro svolto, della comprensione delle soluzioni proposte, della capacità di motivare le scelte effettuate e della correttezza della dichiarazione fornita. L'obiettivo è promuovere un uso trasparente, critico e responsabile degli strumenti di IA come supporto all'apprendimento e allo sviluppo del progetto.

Riferimenti bibliografici

- [1] T. Dunning. The t-digest: Efficient estimates of distributions. *Software Impacts*, 7:100049, 2021. [https://www.softwareimpacts.com/article/S2665-9638\(20\)30040-3/fulltext](https://www.softwareimpacts.com/article/S2665-9638(20)30040-3/fulltext).
- [2] S. Engelhardt. Calculating percentiles on streaming data part 1: Introduction. <https://www.stevenengelhardt.com/2018/03/06/calculating-percentiles-on-streaming-data-part-1-introduction/>, 2018.
- [3] I. Epicoco, C. Melle, M. Cafaro, M. Pulimeno, and G. Morleo. UDDSketch: Accurate tracking of quantiles in data streams. *IEEE Access*, 8:147604–147617, 2020. <https://ieeexplore.ieee.org/iel7/6287639/8948470/09163358.pdf>.
- [4] L. Fernando, H. Bindra, and K. Daudjee. An experimental analysis of quantile sketches over data streams. In *Proc. of 26th Int'l Conf. on Extending Database Technology, EDBT '23*, 2023. https://cs.uwaterloo.ca/~kdaudjee/Daudjee_Sketches.pdf.
- [5] Z. Karnin, K. Lang, and E. Liberty. Optimal quantile approximation in streams. In *Proc. of IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS '16)*, pages 71–78, 2016. <https://ieeexplore.ieee.org/iel7/7781469/7782901/07782919.pdf>.

- [6] C. Masson, J. E. Rim, and H. K. Lee. DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees. *Proceedings of the VLDB Endowment*, 12(12):2195–2205, 2019. <https://www.vldb.org/pvldb/vol12/p2195-masson.pdf>.